[승코의 차근차근 Flutter] 10, Database(Sqflite)(3)

✅ Flutter `sqflite` 완전 마스터: 실전에서 놓치기 쉬운 사소한 문법 & 디테일까지 완벽 정리

📦 1. DB 관련 기본 세팅, 어디까지 자동이고 어디부터 수동인가?

🔷 getDatabasesPath()

```
final dbPath = await getDatabasesPath();
print(dbPath); // 예: /data/user/0/com.example.myapp/databases
```

> 📌 getDatabasesPath()는 Android/iOS에서 내부 DB 저장 경로를 알아서 반환합니다.

> 📌 `Permission`이 **필요 없습니다.** (내부 저장소에 앱 전용 디렉터리)

📦 2. `join()` 함수는 꼭 써야 할까?

```
final path = join(dbPath, 'my_database.db');
```

> ✅ 꼭 쓰세요. OS마다 경로 구분자 (/ or \)가 다르기 때문에 join()이 자동으로 맞춰줍니다.

📦 3. `openDatabase()`의 `version`은 정말 중요해요

```
await openDatabase(
  path,
  version: 1,
  onCreate: _onCreate,
  onUpgrade: _onUpgrade, // 선택적으로 설정
);
```

❗ 버전이 다르면 어떻게 될까?

- `현재 버전 > 기존 버전` → `onUpgrade()` 호출
- `현재 버전 = 기존 버전` → 아무 콜백도 호출 안 됨
- `현재 버전 < 기존 버전` → 오류 발생 (다운그레이드는 불가)

```
Future<void> _onUpgrade(Database db, int oldVersion, int newVersion) async
{
  print("업그레이드: $oldVersion → $newVersion");
  // 예: 새로운 컬럼 추가
  await db.execute('ALTER TABLE todos ADD COLUMN createdAt TEXT');
}
```

📦 4. 테이블 생성 쿼리 문법 디테일

```
CREATE TABLE todos (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  isDone INTEGER NOT NULL DEFAULT 0
)
```

🔍 자주 놓치는 것들

항목	설명
`AUTOINCREMENT`	SQLite에서 `INTEGER PRIMARY KEY` 만 있으면 자동 증가됨. **AUTOINCREMENT는 꼭 필요하지 않음** (하지만 명시하면 더 안전함)
`DEFAULT`	값을 주지 않으면 기본값 사용 (예: `isDone` 이 기본 0)
`TEXT`	문자열, `VARCHAR` 대신 `TEXT` 사용
`BOOLEAN` 없음	대신 `INTEGER(0,1)` 사용

📦 5. insert()의 모든 세부사항

```
await db.insert(
  'todos',
  {
    'title': '청소하기',
    'isDone': 0
  },
  conflictAlgorithm: ConflictAlgorithm.replace,
);
```

🔍 conflictAlgorithm 종류

값	설명
`replace`	같은 PK가 있으면 덮어씀
`ignore`	무시함 (기존 데이터 유지)

값	설명
`abort` (기본값)	충돌 시 오류 발생
`fail`, `rollback`	잘 쓰이지 않음

❗ 왜 toMap()을 써야 할까?

모든 CRUD 함수 (`insert`, `update`, `query`)는 내부적으로 `Map<String, dynamic>` 타입만 받습니다.

따라서 다음과 같은 변환 함수가 필수입니다.

```
Map<String, dynamic> toMap() {
  return {
    'id': id,
    'title': title,
    'isDone': isDone ? 1 : 0,
  };
}
```

📦 6. query()의 모든 파라미터 상세 분석

```
await db.query(
  'todos',
  columns: ['id', 'title'], // 일부 컬럼만 가져오기
  where: 'isDone = ? AND title LIKE ?',
  whereArgs: [0, '%공부%'],
  orderBy: 'id DESC',
  limit: 10,
  offset: 20,
);
```

파라미터	설명
`columns`	가져올 컬럼 목록 (없으면 `*`)
`where`	조건문 (AND/OR 사용 가능)
`whereArgs`	보안 목적 파라미터 (`?` 대체값)

파라미터	설명
`orderBy`	정렬 기준 (`createdAt DESC` 등)
`limit` / `offset`	페이징 (ex. 10개씩 3페이지 → limit: 10, offset: 20)

📦 7. update() 사용 시 가장 많이 하는 실수

```
await db.update(
  'todos',
  {'isDone': 1},
  where: 'id = ?',
  whereArgs: [todo.id],
);
```

❗ 실수 예시

```
await db.update('todos', {'isDone': 1}); // ← 모든 row가 업데이트됨!
```

→ **항상 `where`를 명시해야 함!**

📦 8. rawQuery()는 언제 쓸까?

❌ 잘못된 사용

```
await db.query('SELECT * FROM todos'); // ❌ query()는 SQL 문자열을 받지 않음!
```

✅ 올바른 사용

```
await db.rawQuery('SELECT * FROM todos WHERE isDone = ?', [0]);
```

> rawQuery는 자유도가 높지만 오류 가능성도 높습니다.

> 초보자는 기본 `insert/query/update/delete` 사용을 권장합니다.

📦 9. deleteDatabase()로 초기화

```
String path = join(await getDatabasesPath(), 'todo.db');
await deleteDatabase(path);
```

> 데이터 구조를 바꿨는데 반영이 안 된다면 초기화 필요. 특히 테스트 중 많이 사용됨.

📦 10. Provider, setState와 함께 써야 화면에 반영됨

`sqflite`는 Flutter의 `state`를 자동으로 갱신하지 않음.

```
await db.insert(...);
await loadTodos(); // ← 다시 읽어와야 함
notifyListeners(); // Provider일 경우
setState(() {});   // StatefulWidget일 경우
```

📦 11. 날짜/시간 저장 방법

SQLite는 `DATETIME` 타입이 없습니다.

방법 1: `TEXT` (ISO 형식)

```
String now = DateTime.now().toIso8601String();
// 예: 2025-07-11T15:23:01.000Z
```

방법 2: `INTEGER` (timestamp)

```
int timestamp = DateTime.now().millisecondsSinceEpoch;
```

→ 조회 시 다시 변환:

```
DateTime.fromMillisecondsSinceEpoch(timestamp);
```

📦 12. 데이터 존재 여부 체크

```
final result = await db.query(
  'todos',
  where: 'title = ?',
  whereArgs: ['공부'],
);

bool exists = result.isNotEmpty;
```

📦 13. 트랜잭션 사용법

```
await db.transaction((txn) async {
  await txn.insert(...);
  await txn.update(...);
});
```

> 여러 작업을 하나의 트랜잭션으로 묶어서 안정적인 처리 가능

✅ 마무리 요약: `sqflite` 문법 마스터 레벨 체크리스트

항목	체크
DB 생성 위치 및 구조 이해	✅
테이블 생성 SQL 문법 완벽 이해	✅
insert/query/update/delete 모든 옵션 암기	✅
Map ↔ Object 변환 함수 구현 가능	✅

항목	체크
`where`, `whereArgs`의 보안 역할 이해	✅
`version`과 `onUpgrade()`의 의미 숙지	✅
`Provider` 또는 `setState()`로 UI 연동 가능	✅
rawQuery, 트랜잭션 등의 고급 문법 사용 가능	✅